



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

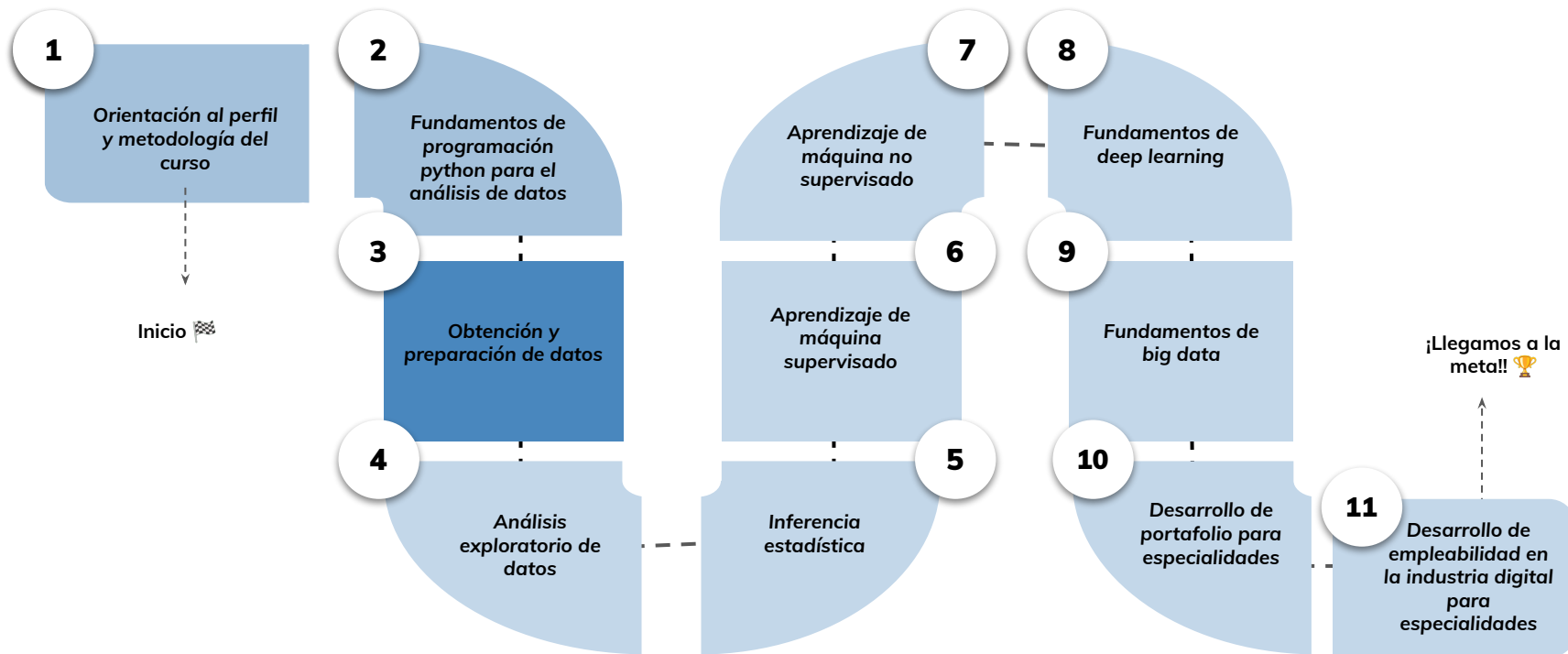


› Data wrangling - Parte I

Aprendizaje Esperado 5: Aplicar técnicas de data wrangling para la manipulación, enriquecimiento y transformación de datos en una estructura de tipo serie o dataframe para la resolución de un problema.

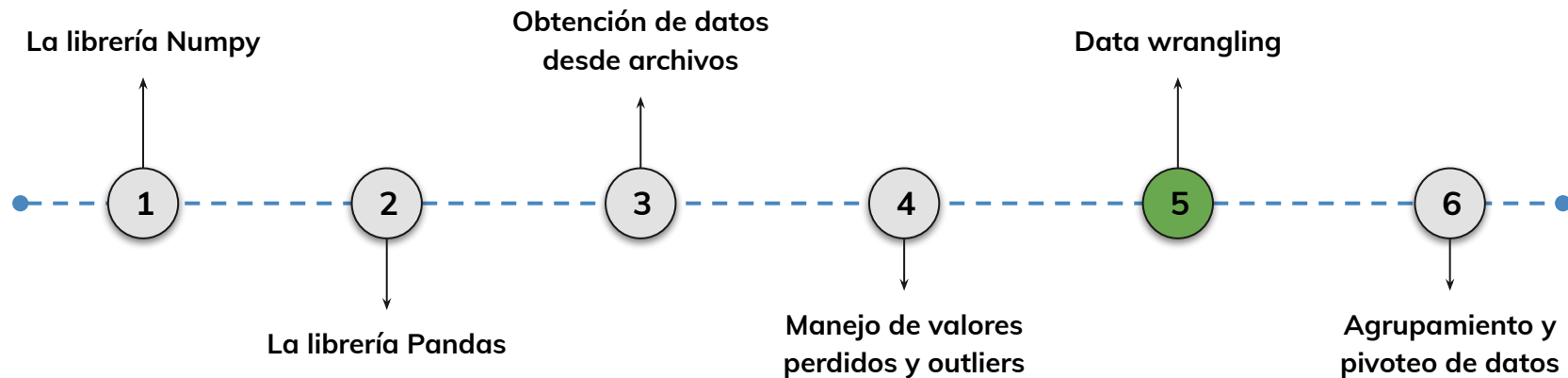
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Transformación de datos con Pandas

Aprenderás a aplicar técnicas de limpieza, estandarización y transformación de datos utilizando Pandas. Esto incluye detectar errores, convertir tipos, eliminar duplicados y enriquecer datasets para dejarlos listos para el análisis.

Limpieza de datos

Enriquecimiento y transformación

Eliminación de duplicados

Reemplazo de valores

Discretización y mapeo

Conversión de tipos y ordenamiento

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender qué es el data wrangling y su rol en el análisis.
- Aplicar técnicas de limpieza, normalización y reestructuración.
- Utilizar Pandas para manipular y transformar datos reales.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Implementamos detección de outliers con Isolation Forest.
- Aplicamos winsorización y transformaciones para reducir su impacto.
- Utilizamos modelos robustos para minimizar el efecto de valores atípicos.
- Integramos tratamiento de valores perdidos y outliers en un único flujo.
- Documentamos el proceso y consideramos implicancias éticas y prácticas.

DATA WRANGLING

DATA WRANGLING

El Data Wrangling es un proceso fundamental en el análisis de datos que permite transformar información cruda en datos estructurados y listos para su análisis.





QUÉ ES DATA WRANGLING

El Data Wrangling es el proceso de transformar y estructurar datos en un formato adecuado para el análisis. Este proceso incluye la limpieza, normalización y reestructuración de los datos para que sean más accesibles y comprensibles. Su objetivo principal es mejorar la calidad de los datos, reducir errores y asegurar que se encuentren en un formato adecuado para su uso en modelos analíticos.

Tareas del Data Wrangling

Conversión de tipos de datos

Ajustar los datos al tipo correcto para su procesamiento.

Corrección de valores erróneos

Identificar y modificar valores incorrectos.

Estandarización de datos

Unificar unidades de medida, formatos de fecha y convenciones de nombres.

Detección y eliminación de valores atípicos

Identificar valores extremos que puedan sesgar el análisis.

Normalización y escalado

Ajustar los valores para que estén en un rango comparable.

Ejemplo de transformación de datos en Pandas

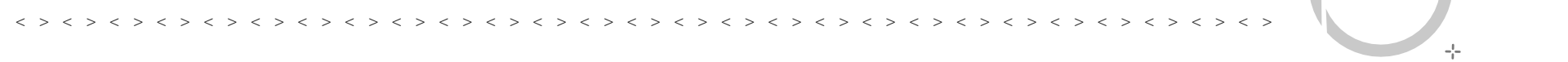
En este ejemplo, se corrigen formatos de fecha inconsistentes y se convierten los precios a valores numéricos.

[illegible]



PRINCIPALES TAREAS DE DATA WRANGLING

El Data Wrangling implica diversas tareas esenciales que permiten mejorar la calidad y estructura de los datos. Algunas de las más importantes incluyen:



Principales tareas de Data Wrangling

1 Ordenamiento y manipulación de datos

Reorganización de los datos para facilitar su análisis.

2 Eliminación de registros duplicados

Identificación y eliminación de registros repetidos que pueden afectar la calidad del análisis.

3 Reemplazo de valores

Sustitución de valores erróneos o inconsistentes para estandarizar la información.

Principales tareas de Data Wrangling (continuación)

1 Discretización y binning

Agrupación de valores continuos en categorías para facilitar la interpretación.

2 Enriquecimiento de datos

Aplicación de funciones y transformaciones para agregar valor a los datos.

3 Manipulación de la estructura de un DataFrame

Ajuste del formato de los datos para mejorar su usabilidad.



ORDENAMIENTO Y MANIPULACIÓN DE DATOS

El ordenamiento y manipulación de datos son procesos fundamentales que permiten organizar la información de manera que sea más fácil de analizar y comprender.



Muestras aleatorias y permutación de la data

El muestreo aleatorio es una técnica utilizada para seleccionar subconjuntos de datos de manera aleatoria. Esto es útil cuando se trabaja con grandes volúmenes de datos y se desea obtener una muestra representativa para el análisis. La permutación, por otro lado, permite reorganizar aleatoriamente los datos en un orden diferente, lo cual es útil en validaciones cruzadas o en experimentos donde se desea eliminar el sesgo del orden original de los datos.

```
import pandas as pd
import numpy as np

# Crear un DataFrame con datos de ejemplo
df = pd.DataFrame({'ID': np.arange(1, 101), 'Valor': np.random.randint(1, 100, 100)})

# Obtener una muestra aleatoria de 10 filas
df_muestra = df.sample(n=10, random_state=42)
print(df_muestra)
```

Ejemplo de muestreo aleatorio en Pandas

En este ejemplo se muestra cómo seleccionar una muestra aleatoria de un DataFrame utilizando el método `sample()` de Pandas.

Permutación aleatoria de datos

También es posible realizar una permutación aleatoria de los datos, es decir, reorganizar el orden de las filas.

Este proceso es útil para mezclar datos antes de realizar entrenamientos de modelos de machine learning o validaciones cruzadas.

```
# Reordenar aleatoriamente todas las filas
df_permutado = df.sample(frac=1, random_state=42)
print(df_permutado.head())
```



El ordenamiento de los datos permite organizar la información según valores específicos. En Pandas, se puede ordenar de forma ascendente o descendente con `sort_values()`.

El ordenamiento de los datos permite organizar la información según valores específicos. En Pandas, se puede ordenar de forma ascendente o descendente con `sort_values()`.

```
# Ordenar por la columna 'Valor' de forma ascendente
df_ordenado = df.sort_values(by='Valor', ascending=True)
print(df_ordenado.head())
```

Ejemplo de ordenamiento por una columna

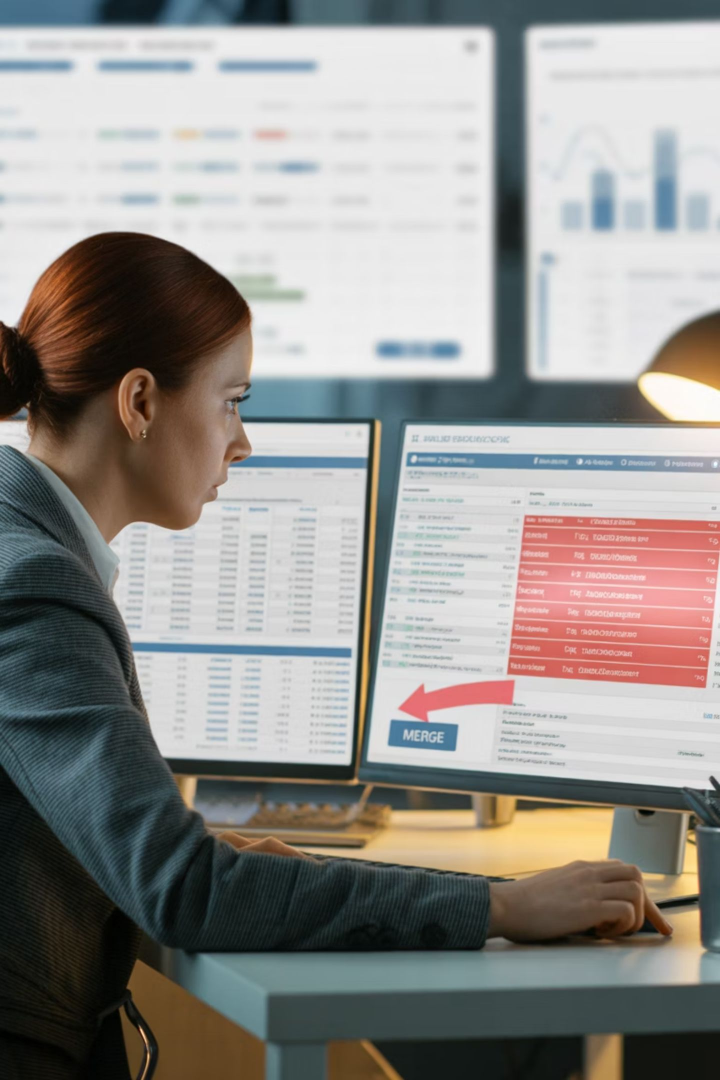
En este ejemplo se muestra cómo ordenar un DataFrame por una columna específica utilizando el método `sort_values()` de Pandas.

```
# Ordenar primero por 'Valor' y luego por 'ID'  
df_ordenado = df.sort_values(by=['Valor', 'ID'], ascending=[True, False])
```

Ordenamiento por múltiples columnas

También es posible ordenar por múltiples columnas.

Este método es útil cuando se necesita priorizar ciertos criterios dentro de un conjunto de datos.



Detección y eliminación de registros duplicados

Los registros duplicados pueden generar sesgos en el análisis. Pandas ofrece métodos sencillos para detectarlos y eliminarlos.

```
# Identificar registros duplicados
duplicados = df.duplicated()
print(duplicados.sum()) # Número de registros duplicados
```

Ejemplo de detección de duplicados

En este ejemplo se muestra cómo detectar registros duplicados en un DataFrame utilizando el método `duplicated()` de Pandas.


```
# Eliminar registros duplicados
df_sin_duplicados = df.drop_duplicates()
print(df_sin_duplicados.shape)
```

Eliminación de duplicados

Para eliminar los registros duplicados, se utiliza el método `drop_duplicates()` de Pandas.

Esto asegura que el conjunto de datos contenga únicamente información única y sin redundancias.

Reemplazo de valores

En ocasiones, los conjuntos de datos contienen valores incorrectos, mal escritos o inconsistentes que deben corregirse. Pandas ofrece funciones para reemplazar valores de manera eficiente.



```
# Crear un DataFrame con valores erróneos
df = pd.DataFrame({'Producto': ['Laptop', 'Tablet', 'Celular', 'Lapto', 'Tablet'],
                  'Precio': [1000, 500, 800, 950, 520]})

# Reemplazar valores incorrectos
df['Producto'].replace({'Lapto': 'Laptop'}, inplace=True)
print(df)
```

Ejemplo de reemplazo de valores

En este ejemplo se muestra cómo reemplazar valores específicos en un DataFrame utilizando el método `replace()` de Pandas.

```
# Reemplazar valores que contengan un patrón específico  
df['Producto'] = df['Producto'].str.replace(r'Lapto', 'Laptop', regex=True)
```

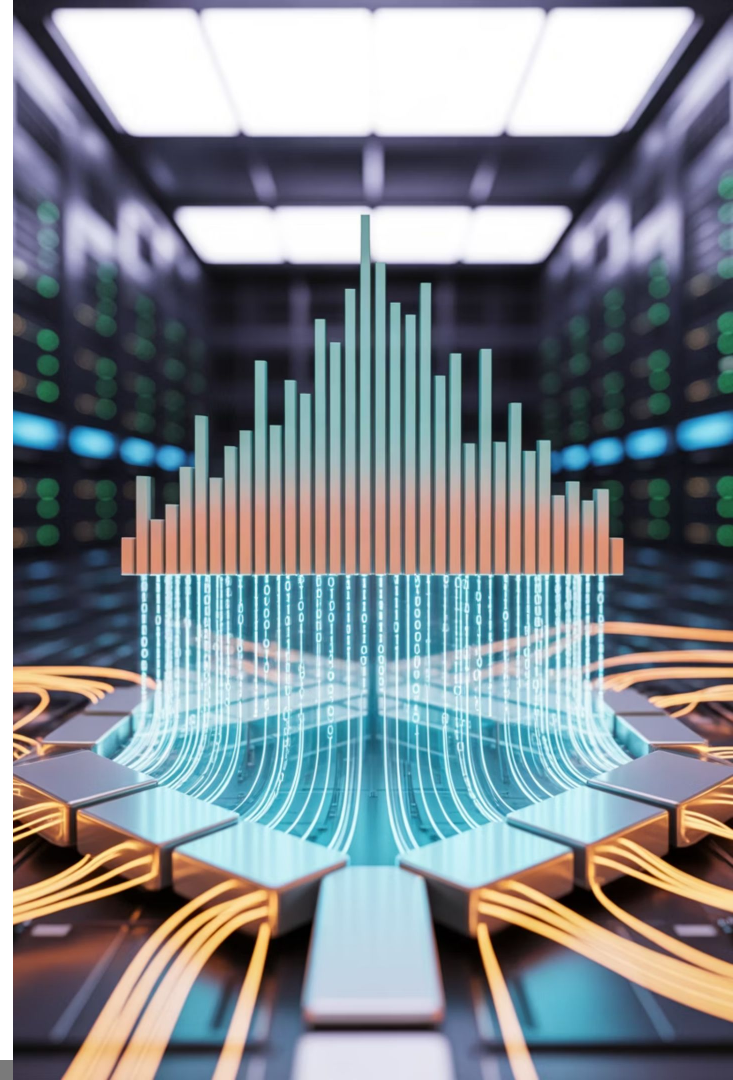
Reemplazo con expresiones regulares

También es posible usar expresiones regulares para reemplazos más avanzados.

El reemplazo de valores es fundamental para estandarizar los datos y evitar errores en el análisis posterior.

Discretización y binning

La discretización es un proceso que convierte valores continuos en categorías o intervalos (bins). Esto es útil cuando se quiere agrupar datos numéricos en rangos específicos.



```
# Crear un DataFrame con datos continuos
df = pd.DataFrame({'Edad': [22, 25, 30, 35, 40, 45, 50, 55, 60]})

# Definir los rangos (bins)
bins = [20, 30, 40, 50, 60]
etiquetas = ['Joven', 'Adulto Joven', 'Adulto', 'Mayor']

# Aplicar discretización
df['Grupo Edad'] = pd.cut(df['Edad'], bins=bins, labels=etiquetas)
print(df)
```

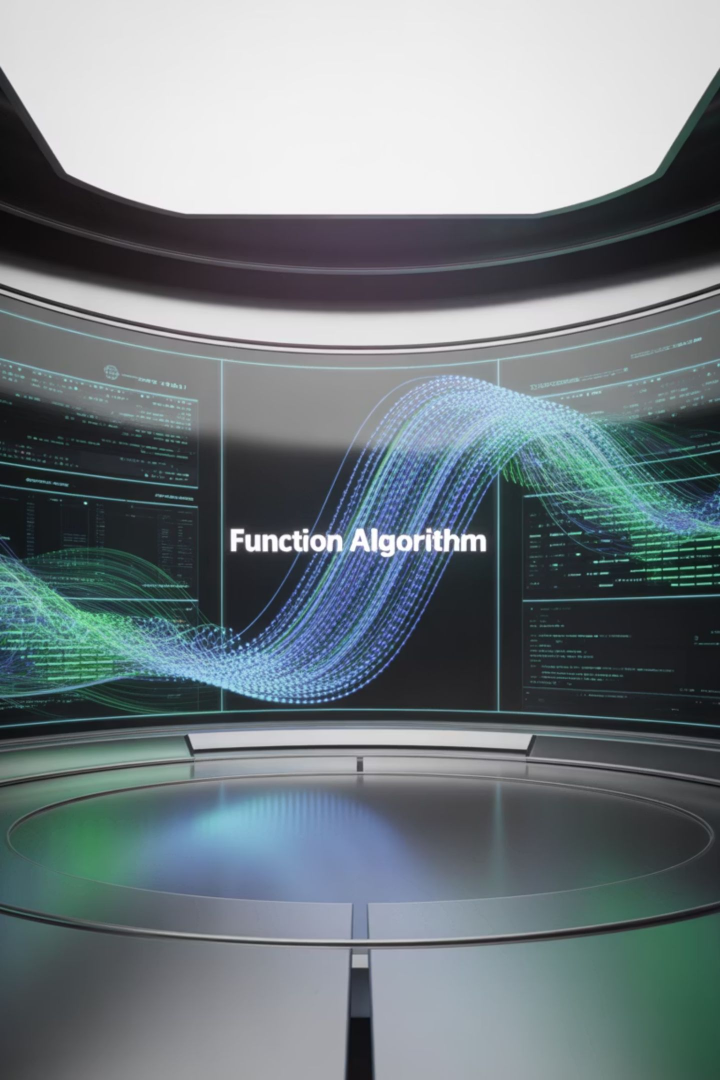
Ejemplo de binning en Pandas

En este ejemplo se muestra cómo convertir valores numéricos continuos en categorías discretas utilizando el método `cut()` de Pandas. Este método facilita la segmentación y agrupación de datos en análisis exploratorios y visualizaciones.



ENRIQUECIMIENTO DE LA DATA

El enriquecimiento de datos implica transformar la información existente para agregar valor al conjunto de datos, mejorando su calidad y utilidad para el análisis.



Utilización de funciones y mapeos

El enriquecimiento de datos implica transformar la información existente para agregar valor al conjunto de datos. Una técnica clave es el uso de funciones y mapeos para modificar columnas o generar nuevas variables en un DataFrame.

Pandas ofrece el método `.map()` para aplicar una función a cada elemento de una serie. Este método es útil cuando se necesita reemplazar valores en una columna o realizar conversiones simples.

Ejemplo de uso de .map() en una columna categórica

En este ejemplo se muestra cómo utilizar el método map() para transformar valores en una columna categórica. El método .map() permite transformar valores en base a reglas definidas, facilitando la categorización y estandarización de datos.

```
import pandas as pd

# Crear un DataFrame
df = pd.DataFrame({'Producto': ['Laptop', 'Tablet', 'Celular'],
                  'Categoría': ['Electrónica', 'Electrónica', 'Comunicación']})

# Crear un diccionario de mapeo
mapeo = {'Electrónica': 'Tecnología', 'Comunicación': 'Telecomunicaciones'}

# Aplicar mapeo
df['Nueva Categoría'] = df['Categoría'].map(mapeo)
print(df)
```

Aplicación de funciones dentro de una serie de datos (apply)

Cuando se requieren transformaciones más complejas, el método `.apply()` es una opción poderosa. Permite aplicar funciones personalizadas a cada valor de una serie.



Ejemplo de `.apply()` en una columna numérica

En este ejemplo se muestra cómo utilizar el método `apply()` para aplicar una función personalizada a cada valor de una columna numérica.

Este método es útil cuando se necesita realizar cálculos en cada valor de una columna.



Aplicación de funciones dentro de un dataframe (apply)

El método `.apply()` también puede utilizarse en un DataFrame completo para aplicar funciones a filas o columnas específicas.

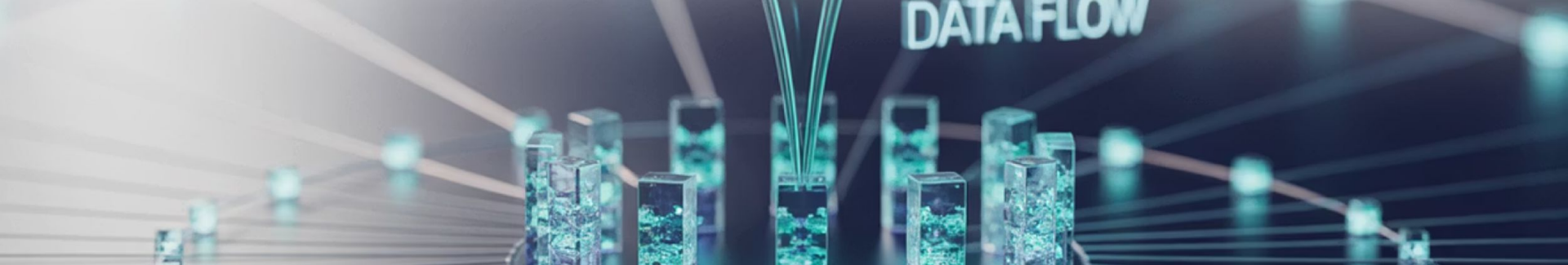
Ejemplo de .apply() en múltiples columnas

En este ejemplo se muestra cómo utilizar el método `apply()` para aplicar una función a múltiples columnas de un DataFrame. Este método permite realizar transformaciones en base a múltiples columnas, enriqueciendo los datos con nueva información derivada.

```
# Crear un DataFrame
df = pd.DataFrame({'Nombre': ['Ana', 'Luis', 'Carlos'],
                  'Edad': [25, 30, 28],
                  'Salario': [50000, 54000, 58000]})

# Aplicar una función a varias columnas
def categorizar_fila(fila):
    if fila['Salario'] > 55000:
        return 'Alto'
    else:
        return 'Medio'

# Aplicar función a cada fila
df['Nivel Salarial'] = df.apply(categorizar_fila, axis=1)
print(df)
```



Utilización de expresiones lambda en la aplicación de funciones

Las expresiones lambda permiten definir funciones en una sola línea y aplicarlas a una columna o DataFrame sin necesidad de definir funciones separadas.

```
# Crear un DataFrame
df = pd.DataFrame({'Precio': [1000, 500, 800, 1500]})

# Aplicar descuento del 10%
df['Precio con Descuento'] = df['Precio'].apply(lambda x: x * 0.90)
print(df)
```

Ejemplo de expresión lambda para calcular descuentos

En este ejemplo se muestra cómo utilizar una expresión lambda para calcular descuentos en un DataFrame. Las funciones lambda son una opción eficiente cuando se requieren transformaciones simples sin necesidad de escribir funciones extensas.



Live Coding

¿En qué consistirá la Demo?

Transformación básica de datos con Pandas

1. Carga de datos desde CSV.
2. Eliminación de duplicados.
3. Reemplazo de valores nulos.
4. Conversión de tipos y ordenamiento.

Tiempo: 20 minutos



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Limpieza y enriquecimiento de un dataset

Limpieza y enriquecimiento de un dataset

Contexto: 🙌

Dado un archivo con registros de ventas desordenados y sucios

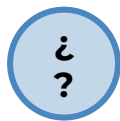
Consigna: ✍️

1. Ordenar por fecha y monto.
2. Reemplazar valores vacíos con “Desconocido”.
3. Crear una nueva columna con categorías según el monto.
4. Renombrar columnas para mayor claridad.

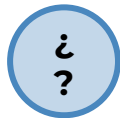
Tiempo 🕒: 30 minutos

Plataforma: Jupyter Notebook / Google Colab

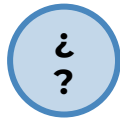
#Momentode Preguntas...



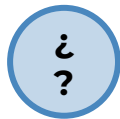
¿Qué implica el proceso de data wrangling?



¿Qué método de Pandas permite eliminar duplicados?



¿Qué función permite aplicar una lógica a cada fila?



¿Qué es la discretización y para qué se usa?

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Identificaste qué implica el proceso de Data Wrangling.**
- ✓ **Aplicaste funciones básicas de Pandas para limpiar datos.**
- ✓ **Utilizaste técnicas como binning, map y apply.**
- ✓ **Comprendiste el valor del enriquecimiento de datos.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

